

Annotations Explained in Easy Words

A guide to every annotation used in **two** projects:

Part A — AshaSathi (github.com/Rahul2-5/AshaSathi)

Part B — StudentSharingPortalProject (github.com/Rahul2-5/StudentSharingPortalProject)

Generated 18 September 2026. Every annotation, count and example below was taken directly from the repository source code, not from memory.

How to use this document

Part A explains all 59 annotations found in AshaSathi. Part B covers StudentSharingPortal: it explains in full the 9 annotations that are unique to that project, and lists the 36 shared ones in a table with their counts and any difference in how they are used. Nothing is skipped — every annotation in either repository appears somewhere in this document.

What is an annotation?

An annotation is a small label starting with @ that you stick on a class, a method, a field or a parameter. It does not run like normal code. It is a note that a framework (Spring Boot), a code generator (Lombok), or the compiler reads, and then does extra work for you. Think of it as a sticky note: *“treat this class as a database table”*, *“call this method when a web request arrives”*, *“write the getters and setters for me”*.

What this project is built with

Java 21, Spring Boot 3.5.7 (web, security, data-jpa, webflux, validation), Lombok, PostgreSQL / SQLite, plus a Flutter (Dart) mobile app and a small Python FastAPI OCR service. The Java code sits in two copies of the same application: src/ (49 files) and Backend/src/ (33 files). The counts below cover both copies together.

PART A — AshaSathi

Java 21 · Spring Boot 3.5.7 · Lombok · PostgreSQL/SQLite · Flutter app · Python OCR service. 59 distinct annotations.

1. Starting and configuring the application

@SpringBootApplication (2 uses · main class)

The one label that starts everything. It says: this is the entry class, scan this package for other annotated classes, and auto-configure the web server, database and security based on the libraries found.

In this project: @SpringBootApplication on AshaSathiApplication.java

@Configuration (7 uses · config classes)

Marks a class as a settings class. Spring reads it once at startup and keeps whatever objects it hands back.

In this project: @Configuration on SecurityConfig, AsyncConfig and the Gemini AI client config

@Bean (10 uses · methods inside @Configuration)

Put on a method whose return value should be stored and reused everywhere in the app. A 'bean' is simply a ready-made object Spring keeps for you. You can give it a name so you can ask for that exact one later.

In this project: @Bean("geminiWebClient") and @Bean(name = "medicalProcessingExecutor")

@Component (6 uses · helper classes)

The generic version of @Service / @Repository. It says: build one of these at startup and keep it ready for anyone who needs it.

In this project: @Component on the JWT filter and utility classes

@Service (16 uses · service classes)

Same effect as @Component, but it labels the class as business logic — the code that actually does the work (save a patient, summarise a document).

In this project: @Service on PatientService, DocumentService and similar classes

@Repository (10 uses · database interfaces)

Same effect again, but it marks database access code. It also converts messy low-level database errors into cleaner Spring exceptions.

In this project: @Repository on the JPA repository interfaces such as PatientRepository

@Value (16 uses · fields)

Pulls a setting out of application.properties instead of hard-coding it. The part after the colon is the default if the setting is missing.

In this project: @Value("\${app.jwt.secret}") and @Value("\${app.jwt.expiration-ms:86400000}") — the second defaults to 86,400,000 ms (24 hours) if not configured

@Qualifier (1 uses · injection point)

When two beans of the same type exist, this says which one you want, by name.

In this project: @Qualifier("geminiWebClient") — pick that specific WebClient, not any other

@RequiredArgsConstructor (12 uses · classes (Lombok))

Lombok writes a constructor for you that takes all the final fields. Spring then uses that constructor to hand the class its dependencies, so you never write 'new' or @Autowired.

In this project: @RequiredArgsConstructor on controllers and services with final repository fields

@Autowired (written in full) (1 uses · one field)

The older way to ask Spring to inject a dependency into a field. Used once here in its fully-qualified form; constructor injection via Lombok is used everywhere else and is the preferred style.

In this project: `@org.springframework.beans.factory.annotation.Autowired`

@PostConstruct *(2 uses · methods)*

Run this method once, automatically, right after the object is built and its dependencies are filled in. Good for warm-up or validation of settings.

2. The web layer — receiving HTTP requests

@RestController (7 uses · controller classes)

Marks a class that answers web requests, and says every returned object should be turned into JSON automatically (no HTML pages).

In this project: @RestController on AuthController, PatientController, etc.

@RequestMapping (6 uses · controller classes)

Sets the common URL prefix for every method in the class.

In this project: @RequestMapping("/api/auth"), @RequestMapping("/api/families"), @RequestMapping("/api/medical-documents")

@GetMapping (14 uses · methods)

Run this method when a GET request (fetch/read data) arrives at the given path.

In this project: @GetMapping, @GetMapping("/") — combined with the class prefix to form the full URL

@PostMapping (9 uses · methods)

Run this method for a POST request (create something / submit data).

In this project: @PostMapping("/google") — the Google sign-in endpoint

@PutMapping (2 uses · methods)

Handles PUT: replace an existing record completely.

In this project: @PutMapping("/{id}")

@PatchMapping (1 uses · method)

Handles PATCH: change only part of a record, not the whole thing.

In this project: @PatchMapping("/{id}/assign") — assign a task without rewriting the rest of it

@DeleteMapping (5 uses · methods)

Handles DELETE: remove a record.

In this project: @DeleteMapping("/{id}")

@PathVariable (12 uses · method parameters)

Takes a value out of the URL itself and puts it into the method parameter. In /api/tasks/7 the 7 lands in the parameter.

In this project: @PathVariable Long id, paired with @PutMapping("/{id}")

@RequestParam (2 uses · method parameters)

Takes a value from the query string (the part after ?) or a form field. required = false means the request still works if it is missing.

In this project: @RequestParam("file") and @RequestParam(value = "patientId", required = false)

@RequestBody (9 uses · method parameters)

Takes the JSON sent in the request body and converts it into a Java object automatically.

In this project: @RequestBody on DTO parameters in the POST and PUT endpoints

@RequestPart (1 uses · method parameter)

Used for multipart uploads (a file plus other fields in one request). Picks out one named part — here, the uploaded image.

In this project: `@RequestPart("photo")`

@CrossOrigin *(2 uses · controllers)*

Allows a web page served from another address (the Flutter web app) to call this API. Browsers block that by default. `maxAge` is how long the browser may cache that permission.

In this project: `@CrossOrigin(origins = "*", maxAge = 3600)` — any origin allowed, cached for 3600 s (1 hour). Note: "*" is convenient in development but is wide open in production.

3. Security

@EnableWebSecurity (2 uses · security config)

Switches on Spring Security's web filter chain so the config class can define who may reach which URL.

@EnableMethodSecurity (1 uses · security config)

Allows security rules to be written on individual methods, not just on URLs. `prePostEnabled = true` is what makes `@PreAuthorize` work.

In this project: `@EnableMethodSecurity(prePostEnabled = true)`

@PreAuthorize (12 uses · controller methods)

Checks the rule in quotes BEFORE the method runs. If the logged-in user fails the check, the method never executes and the caller gets 403 Forbidden.

In this project: `@PreAuthorize("hasAnyRole('USER', 'ADMIN', 'ASHA')")` — any of the three roles may call it; `@PreAuthorize("isAuthenticated()")` — just needs to be logged in

4. Database entities (JPA / Hibernate)

@Entity (11 uses · model classes)

Says this class maps to a database table. One object of the class = one row.

@Table (11 uses · entity classes)

Sets the real table name, and can also declare indexes.

In this project: `@Table(name = "families")`, `@Table(name = "drug_master")`, and `@Table(name = "ocr_line", indexes = { ... })`

@Index (1 uses · inside @Table)

Creates a database index on a column so lookups on that column are fast.

In this project: `@Index(name = "idx_ocr_line_document", columnList = "document_id")`

@Id (11 uses · one field per entity)

Marks the primary key — the field that uniquely identifies a row.

@GeneratedValue (11 uses · the @Id field)

Lets the database create the key value. IDENTITY means an auto-increment column: 1, 2, 3...

In this project: `@GeneratedValue(strategy = GenerationType.IDENTITY)`

@Column (76 uses · entity fields)

Describes the column behind a field: its name, length, whether it may be null, or the raw SQL type. The most-used annotation in this project.

In this project: `@Column(length = 15)` — e.g. a phone number; `@Column(columnDefinition = "TEXT")` — long free text such as an AI summary

@Enumerated (2 uses · enum fields)

Tells JPA how to store an enum. `EnumType.STRING` saves the readable name ('ASHA') instead of a number, so the database stays readable and safe when the enum order changes.

In this project: `@Enumerated(EnumType.STRING)`

@ManyToOne (6 uses · entity fields)

Many rows of this table point at one row of another table (many documents belong to one patient). `fetch = LAZY` means do not load the other object until it is actually used — faster. `optional = false` means the link may not be empty.

In this project: `@ManyToOne(fetch = FetchType.LAZY, optional = false)`

@JoinColumn (6 uses · next to @ManyToOne)

Names the foreign-key column that stores the link.

In this project: `@JoinColumn(name = "family_id", nullable = false), @JoinColumn(name = "document_id", nullable = false)`

@OneToMany (3 uses · collection fields)

The other side of the link: one parent holds a list of children. `mappedBy` names the field on the child that owns the relationship. `cascade = ALL` passes saves and deletes down to the children; `orphanRemoval = true` deletes a child that is taken out of the list.

In this project: `@OneToMany(mappedBy = "document", cascade = CascadeType.ALL, orphanRemoval = true)`

@PrePersist (2 uses · entity methods)

Run this method automatically just before the row is inserted the first time — typically to stamp `createdAt`.

@PreUpdate (4 uses · entity methods)

Run automatically just before an existing row is updated — typically to refresh `updatedAt`.

@PersistenceContext (1 uses · field)

Injects the JPA `EntityManager`, the low-level object used to run queries by hand when a repository method is not enough.

5. Repository queries and transactions

@Query (2 uses · repository methods)

Write the query yourself (in JPQL, which works on class and field names rather than table and column names) instead of relying on the method name.

In this project: `@Query("SELECT d FROM DrugMaster d WHERE d.active = true")`

@Modifying (2 uses · repository methods)

Required on a `@Query` that changes data (UPDATE / DELETE). Without it Spring assumes the query only reads and the statement fails.

In this project: `@Modifying with @Query("UPDATE MedicalDocument d SET d.aiSummary = :aiSummary ... WHERE d.id = :id")`

@Param (3 uses · query parameters)

Connects a method parameter to a named placeholder inside the `@Query` string.

In this project: `@Param("aiSummary"), @Param("ashaActions"), @Param("id")`

@Transactional (22 uses · services and repositories)

Wraps the method in one database transaction: either every change inside it succeeds, or all of them are rolled back. `readOnly = true` is a hint for pure reads and lets the database skip change-tracking work.

In this project: `@Transactional` and `@Transactional(readOnly = true)`

6. Background work

@EnableAsync (2 uses · config class)

Turns on background-thread support for the whole application. Without it, `@Async` is ignored.

@Async (3 uses · methods)

Run this method on a separate thread and return immediately. Used so a slow job (OCR and AI summarising of a medical document) does not keep the user waiting for the HTTP response.

In this project: `@Async` on the document-processing methods, running on the `medicalProcessingExecutor` bean

7. JSON conversion (Jackson)

@JsonProperty (14 uses · DTO fields)

Renames a field in the JSON. Java uses camelCase (`documentType`) while the AI service and mobile app use snake_case (`document_type`); this maps between them.

In this project: `@JsonProperty("document_type"), @JsonProperty("doctor_name"), @JsonProperty("asha_actions")`

@JsonIgnore (2 uses · fields)

Keep this field out of the JSON completely. Used for things that must never leave the server (such as a password hash) or that would cause an endless loop between linked entities.

8. Lombok — annotations that write code for you

Lombok generates real Java code at compile time. The methods exist in the compiled class; they are simply not typed out in the source file.

@Data (37 uses · entities and DTOs)

The all-in-one: generates getters, setters, toString(), equals(), hashCode() and a constructor for the final fields. One line replaces roughly a hundred.

@Getter (1 uses · class/field)

Generates only the get methods.

@Setter (1 uses · class/field)

Generates only the set methods.

@NoArgsConstructor (9 uses · entities and DTOs)

Generates an empty constructor. JPA and Jackson both need one to create the object before filling in its fields.

@AllArgsConstructor (9 uses · entities and DTOs)

Generates a constructor with one parameter for every field.

@Builder (7 uses · classes)

Generates a builder so objects can be created readably: Patient.builder().name("x").age(30).build(). Useful when a class has many fields and positional constructor arguments get confusing.

@Slf4j (15 uses · classes)

Creates a ready-to-use logger field called 'log', so you can write log.info(...) or log.error(...) without declaring anything.

9. Plain Java and testing

@Override (14 uses · methods)

Tells the compiler this method is meant to replace one from a parent class or interface. If the name or parameters do not match, compilation fails — it catches typos.

@SuppressWarnings (7 uses · methods and fields)

Silences a specific compiler warning the author judged safe. Use sparingly: it hides the warning, it does not fix the cause.

In this project: @SuppressWarnings("unchecked"), @SuppressWarnings("unused")

@SpringBootTest (1 uses · test class)

Starts the whole Spring application inside the test so real beans and the real wiring are exercised.

@ActiveProfiles (1 uses · test class)

Loads a different set of settings for the test run — typically an in-memory database instead of the real one.

In this project: @ActiveProfiles("test")

@Test (1 uses · test method)

JUnit's mark for 'this method is a test; run it'.

10. Dart annotations (the Flutter mobile app)

@override (206 uses · Dart methods)

Same idea as Java's @Override: this method replaces one from the parent widget or class. Very common in Flutter because every widget overrides build().

@freezed (4 uses · Dart classes)

Marks a class for the Freezed code generator, which writes an immutable data class with copyWith(), equality and JSON support.

@JsonSerializable (3 uses · Dart classes)

Asks the generator to write the toJson() and fromJson() methods for this class.

@JsonKey (23 uses · Dart fields)

Controls how one field maps to JSON — including leaving it out entirely.

In this project: @JsonKey(includeFromJson: false, includeToJson: false) — a local-only field that never travels over the network

@Default (8 uses · Freezed constructor parameters)

Gives a field a default value when the incoming JSON does not contain it.

In this project: @Default(false), @Default(true), @Default('') , @Default({})

@pragma (14 uses · Dart methods)

A direct instruction to the Dart compiler (for example, do not remove this entry point). Mostly found in generated code.

@useResult / **@nodoc** (8 / 23 uses · Dart)

@useResult warns if you call a method and ignore what it returns. @nodoc hides generated members from the documentation tool. Both appear in generated files.

11. Python decorators (the FastAPI OCR service)

Python calls them decorators rather than annotations, but they work the same way — a label above a function that changes how it is used.

@app.post(...) (3 uses · FastAPI routes)

Registers the function as the handler for a POST URL. response_model tells FastAPI the shape of the reply, and it validates and documents it automatically.

In this project: @app.post("/extract-text", response_model=OCRExtractionResponse); also "/parse-text" and "/parse-and-summarize"

@app.get(...) (1 uses · FastAPI route)

Same for a GET URL.

In this project: @app.get("/health") — the liveness check

@app.on_event(...) (1 uses · startup hook)

Run this function once when the service starts — here, to load the OCR model before the first request arrives.

In this project: @app.on_event("startup")

PART B — StudentSharingPortalProject

Java 17 · Spring Boot 3.2.3 · Lombok · MySQL · JWT 0.11.5 · Apache Tika 2.9.2 (text extraction/OCR) · React frontend (.jsx). 45 distinct annotations, all in Java — the React frontend uses none, because JavaScript has no annotations.

Code layout: Backend/StudentSharingPortal/src, split into Config, Controller, DTO, Entity, JWT, Repository, Services, plus a real test folder (5 test classes). Unlike AshaSathi there is only one copy of the source, so the counts here are true counts.

B1. The 9 annotations that appear here and NOT in AshaSathi

@Autowired (26 uses · fields and constructors)

Asks Spring to hand this class the object it needs, instead of creating it with 'new'. This is the classic field-injection style. AshaSathi mostly replaces it with Lombok's @RequiredArgsConstructor; this project writes it out by hand in 12 files. Both work; constructor injection is the style Spring itself now recommends, because it makes dependencies visible and the class testable without a framework.

In this project: @Autowired on repository and service fields across controllers and services

@CreationTimestamp (3 uses · entity date fields)

A Hibernate annotation: fill this field with the current date and time automatically when the row is first saved. It replaces the manual @PrePersist method AshaSathi uses for the same job.

In this project: @CreationTimestamp on the uploadedAt / createdAt fields of the entities

@UniqueConstraint (1 uses · inside @Table)

Tells the database that a combination of columns may not repeat. Here it stops one student from rating the same study material twice — the rule is enforced by the database itself, so it holds even if the Java check is bypassed.

In this project: @Table(name = "ratings", uniqueConstraints = @UniqueConstraint(columnNames = {"material_id", "rater_id"}))

@AuthenticationPrincipal (5 uses · controller method parameters)

Injects the currently logged-in user straight into the method parameter, taken from the validated JWT. It saves you from reading the security context by hand, and means an endpoint cannot accidentally act on someone else's identity sent in the request body.

In this project: @AuthenticationPrincipal on upload, my-materials and rating endpoints

@ExtendWith (3 uses · test classes)

Plugs an extension into JUnit 5. MockitoExtension is what makes @Mock and @InjectMocks actually get filled in before each test.

In this project: @ExtendWith(MockitoExtension.class)

@Mock (7 uses · test fields)

Creates a fake version of a dependency (a repository, an external API client). The fake returns whatever the test tells it to, so the test runs fast and does not touch a real database or network.

@InjectMocks (3 uses · test fields)

Creates the real class under test and pushes all the @Mock objects into it. One line instead of wiring the object by hand.

In this project: @InjectMocks on the service being tested, e.g. the AI summary service

@BeforeEach (1 uses · test method)

Run this setup method before every single test in the class, so each test starts from a clean, identical state.

@TempDir (written in full) *(3 uses · test method parameters)*

JUnit creates a throwaway folder for the test and deletes it afterwards. Used here for the file-upload and OCR tests, so test files never pile up on disk.

```
In this project: @org.junit.jupiter.api.io.TempDir Path tempDir &mdash; in extractsTextFromPdf  
and extractsTextFromImageViaOcr
```

B2. The 36 shared annotations — counts and differences

These mean exactly what Part A says they mean. Only the count and any noteworthy difference in this project are given.

Annotation	Uses	Note for this project
@Column	14	Includes @Column(columnDefinition = "LONGTEXT") for the stored AI summary, and @Column(length = 1000).
@RequestParam	12	Heavily used here: @RequestParam("file"), ("category"), search keywords.
@Data	9	Same Lombok all-in-one, on entities and DTOs.
@Builder	9	Used in tests too: User.builder().id(1L).email(...).role("STUDENT").build().
@Value	7	Reads \${gemini.api.key}, \${file.upload-dir}, \${ai.summary.max-input-chars}.
@Service	7	7 service classes.
@Test	14	Real test suite here (AshaSathi has 1 test).
@PathVariable	6	e.g. @GetMapping("/download/{id}").
@NoArgsConstructor	6	Same as Part A.
@Bean	6	Plain @Bean only; no named beans.
@AllArgsConstructor	6	Same as Part A.
@PostMapping	5	"/login", "/register", "/upload".
@Param	5	Binds "category", "keyword", "material" into @Query.
@GetMapping	5	"/my", "/download/{id}" and the list endpoint.
@Query	4	Aggregate queries: SELECT AVG(r.score), SELECT COUNT(r) for ratings.
@Table	3	"users", "study_materials", "ratings".
@RestController	3	AuthController, StudyMaterialController, AdminController.
@RequestMapping	3	"/api/auth", "/api/materials", "/api/admin/materials".
@Repository	3	3 repository interfaces.
@Override	3	Same as Part A.
@ManyToOne	3	Both fetch types used: LAZY and EAGER. EAGER always loads the linked row - convenient, but it can trigger many extra queries.
@JoinColumn	3	"uploader_id", "material_id", "rater_id", all nullable = false.
@Id	3	3 entities: User, StudyMaterial, DocumentRating.
@GeneratedValue	3	IDENTITY strategy, same as Part A.
@Entity	3	3 entities.
@CrossOrigin	3	Written bare: @CrossOrigin, with no origins or maxAge set - defaults to allowing all origins.
@Configuration	3	Security, CORS and app config.
@Component	3	Includes the JWT filter.
@RequestBody	2	Login and register payloads.
@Enumerated	2	EnumType.STRING, same as Part A.
@SuppressWarnings	1	"unchecked".
@SpringBootTest	1	The context-loads test.
@SpringBootApplication	1	One entry class (AshaSathi has 2 because of its duplicated source tree).
@PutMapping	1	Update a material.
@EnableWebSecurity	1	Same as Part A.

Annotation	Uses	Note for this project
@DeleteMapping	1	Delete a material.

Things worth fixing in Part B's code

- **@CrossOrigin with no arguments** on both controllers allows requests from any website. Set origins to your React app's address before deploying.
- **No validation annotations** (@Valid, @NotBlank, @Email) anywhere, so login, register and upload payloads reach the service layer unchecked.
- **@Autowired on fields** in 12 files. Switching to constructor injection (or Lombok's @RequiredArgsConstructor, already used in the other project) makes these classes testable without Spring.
- **@ManyToOne(fetch = FetchType.EAGER)** loads the linked row on every query. On a material list this multiplies queries; LAZY plus an explicit fetch join is usually safer.

How to read the counts

Counts were produced by scanning every .java, .dart and .py file in each repository and tallying each @Name occurrence, including ones written inline on method parameters. In Part A the Java application exists in two copies (src/ and Backend/src/), so a count of roughly double is expected for anything appearing once per copy; Part B has a single source tree and its counts are exact. Text like "user@test.com" inside test data was excluded — it is an email address, not an annotation.